



Security Assessment



Aave Stable Vaults

April 2026

Prepared for Aave Labs

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	4
Assessment Methodology	6
Threat and Security Overview	7
Findings Summary	10
Severity Matrix	10
Detailed Findings	11
Low Severity Issues	12
L-01: Stale previewRedeem in trusted strategies can mask deposit slippage, mis-recording rounding loss as principal	12
Informational Severity Issues	14
I-01: Constant STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE will prevent interaction with strategies once share rate hits 10	14
I-02: Transfer of Full Balance May Revert Due to Remaining Shares Redeemability Check	16
Disclaimer	18
About Certora	18

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave Stable Vaults	https://github.com/aave/stable-vault	c9ee565	Solidity

Project Overview

This document describes the security review of **AAVE Stable Vault**. The work was undertaken from **April 27th, 2026** to **May 5th, 2026**.

The following contract list is included in our scope:

```
stable-vault/src/bridging/ccip/CcipAdapter.sol
stable-vault/src/bridging/BaseBridgeAdapter.sol
stable-vault/src/bridging/BridgeParamsCodec.sol
stable-vault/src/core/accounting/AccountingChainGateway.sol
stable-vault/src/core/accounting/StableVault.sol
stable-vault/src/core/accounting/FundsHandler.sol
stable-vault/src/core/earning/EarningChainGateway.sol
stable-vault/src/core/ious/IouToken.sol
stable-vault/src/core/ious/IouTokenManager.sol
stable-vault/src/core/Allocator.sol
stable-vault/src/core/BaseChainGateway.sol
stable-vault/src/core/LocalBalanceAggregator.sol
stable-vault/src/libraries/AssetLib.sol
stable-vault/src/libraries/MathLib.sol
stable-vault/src/misc/Multicall.sol
stable-vault/src/misc/RescuableNative.sol
stable-vault/src/misc/RescuableToken.sol
stable-vault/src/misc/TransferHelperClient.sol
```

```
stable-vault/src/periphery/AssetRegistry.sol
stable-vault/src/periphery/Swapper.sol
stable-vault/src/periphery/TransferHelper.sol
stable-vault/src/periphery/WithdrawalPolicy.sol
stable-vault/src/periphery/OwnedMulticall.sol
stable-vault/src/periphery/EarningChainStateSchemaV1.sol
stable-vault/src/periphery/EarningChainStateProvider.sol
stable-vault/src/types/Constants.sol
stable-vault/src/types/Errors.sol
stable-vault/src/oracles/balance/ChainBalanceOracle.sol
stable-vault/src/oracles/balance/ChainlinkChainBalanceOracleAdapter.sol
stable-vault/src/oracles/balance/ChainlinkL2ChainBalanceOracleAdapter.sol
stable-vault/src/oracles/common/L2ChainlinkOracleAdapter.sol
stable-vault/src/oracles/price/ChainlinkL2PriceOracleAdapter.sol
stable-vault/src/oracles/price/ChainlinkPriceOracleAdapter.sol
stable-vault/src/oracles/price/PriceOracle.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Stable Vault is a semi-fixed rate vault protocol developed by Aave Labs. It allows users to deposit assets into "SubVaults" that offer a specific per-second interest rate, enabling predictable yield generation. Deposited funds are allocated to yield-generating strategies (e.g. Aave). The system tracks each user's original principal independently from accrued interest. In the event the vault cannot meet its total interest obligations, users are prioritized to withdraw at least their original deposit amount.

The system allows users to deposit whitelisted USD denominated stable coins on the accounting (canonical) chain. The depositor is minted vault shares, which are transferable; funds are automatically deployed to the default strategy, when set.

The withdrawal process has two steps:

1. Vault share holders first request withdrawal of principal and interest – they are only allowed to withdraw interest if the protocol is solvent. Withdrawal requests burn vault shares and mint IOU tokens. This stops interest accrual and locks the USD value of the withdrawal.

2. IOU token holders can then redeem it for whitelisted USD denominated stable coins, minus a withdrawal fee. This can be done on the canonical chain; or IOUs can be bridged to any of the earning chains and redeemed there. There is a default withdrawal fee charged from all withdrawals – this fee can be overridden if the user provides a signed message from the whitelisted signer. The withdrawal attempts to use idle funds not deployed to any strategy; if insufficient, it attempts to withdraw from the default strategy; if insufficient, from other strategies.

The system considers one unit of whitelisted stable coins to be worth 1 USD, and does not track deposit per asset back to users.

Deposits accrue a fixed-rate yield, set by governance for each user.

Governance whitelisted addresses can manage the funds, swapping the tokens held; or rebalancing the strategies to which these tokens are deployed.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on logic errors and vulnerabilities in the smart contract code, including accounting-chain and earning-chain interactions, vault share and IOU accounting consistency, interest accrual correctness, defensive rounding, ERC-4626 strategy integration, withdrawal solvency, and asset trust/distrust handling. The review was conducted through manual analysis, invariants, and hypotheses.

The following are treated as trust assumptions, that must hold for the protocol to operate safely:

- Blockchain liveness and integrity: blocks are produced continuously, valid transactions are accepted, invalid ones rejected, fees remain reasonable.
- Bridge and cross-chain messaging integrity: the underlying messaging layers deliver messages reliably and in order, and do not drop, forge, or maliciously reorder cross-chain calls. The audit verified the protocol's integration with these layers only, not their internal security.
- Oracle correctness: the price feeds used for peg and depeg detection are timely, accurate, and manipulation-resistant. The audit verified integration only, not feed quality.
- Strategy integrity: external yield strategies (e.g., Aave) conform to the ERC-4626 standard, remain solvent, and are not malicious or arbitrarily upgradeable. The audit verified the vault's interaction with strategies, not the strategies themselves.
- Admin and governance integrity: multisig keys are secure, the execution time-lock is honored, and governance proposals are not malicious.
- Manager honesty within bounds: the semi-trusted manager performs rebalancing, rate-setting, surplus claims, and cross-chain fund movement without abusing its privileges. It is assumed unable to claim principal or cause principal loss, but capable of affecting yield and operational outcomes.
- Signer integrity: the whitelisted fee-discount signer's key is secure and is used only to sign legitimate withdrawal-fee overrides.
- Off-chain monitoring and response: monitoring systems detect depegs, anomalous balances, and malicious strategies, and operators act promptly, since deposits and withdrawals are permissionless and can deplete value before manual intervention.

System Invariants

The system is designed to preserve the following invariants under all conditions:

- Users can always redeem IOUs for their Original Deposit.
- IOUs are minted for interest only up to trusted surplus above guaranteed obligations.
- Admin may claim surplus only, defined as total assets minus (total principal deposits + total interest accrued).
- TVL cannot be artificially inflated.

Roles

The following roles carry distinct trust levels that must hold for the protocol to operate safely:

- Admin: fully trusted entity operating through a multisig account with an execution time-lock on each operation.
- Manager: semi-trusted entity responsible for rebalancing, setting interest rates, claiming surplus, and moving funds across chains, but unable to claim principal or cause principal loss.

Security Considerations and Threat Analysis

Deposit Integrity. A deposit must never degrade system solvency or shift costs onto existing users. Threats include:

- Deposits that harm the system or other depositors under any scenario.
- Deposit of depegged assets in the absence of a price check, rendering the system insolvent.
- Deployment slippage to the default strategy borne by the vault rather than the depositor.

Denial of Service. Critical actions, especially withdrawals, must remain executable at all times. Threats include:

- Blocking, stalling, or forcing retries of withdrawals or other critical flows.
- Nonce or replay-based DoS of discounted-fee withdrawals (e.g., front-running to consume a single-use signed message).

Access Control. Only authorized roles may perform privileged actions. Threats include:

- Bypassing access restrictions on protected functions.
- Spoofing or replaying signed messages and permissions.

Cross-Chain Integrity. Cross-chain accounting and messaging must be tamper-resistant and reliable. Threats include:

- Manipulating total TVL via cross-chain snapshots.
- Spoofing an earning chain's reported TVL or maliciously minting IOUs through cross-chain components.
- Cross-chain transfers or messages that can be blocked, stuck, or maliciously retried.

TVL and Interest Manipulation. TVL and accrued interest must reflect the true economic state. Threats include:

- Artificial inflation or deflation of TVL, impacting principal and interest claims.
- Manipulating the interest-rate mechanism so the effective rate diverges from the intended rate.

User-to-User Harm. No user action should harm other users or elevate systemic risk. Threats include:

- Actions that damage other users' claims or increase the risk of TVL loss.

Asset and Strategy Risk Management. The protocol must be able to retire dangerous components. Threats include:

- Inability to remove a heavily depegging asset or a malicious strategy to limit risk and losses.
- Limited admin-controlled mechanisms to minimize exposure when an asset or strategy becomes dangerous.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	1	1	1
Informational	2	2	0
Total	3	3	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
L-01	Stale previewRedeem in trusted strategies can mask deposit slippage, mis-recording rounding loss as principal	Low	Fixed
I-01	Constant STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE will prevent interaction with strategies once share rate hits 10	Informational	Acknowledged
I-02	Transfer of Full Balance May Revert Due to Remaining Shares Redeemability Check	Informational	Acknowledged

Low Severity Issues

L-01: Stale previewRedeem in trusted strategies can mask deposit slippage, mis-recording rounding loss as principal

Severity: Low	Impact: Low	Likelihood: Low
Files: src/core/Allocator.sol	Status: Fixed	

Description:

On `allocator::deposit()`, when a strategy's `previewRedeem` reads a stale stored index (not computing the accrue on the go), `balanceBefore` in `_depositToStrategy` is stale and it will not include the accrued amount. The deposit then triggers interest accrual in the strategy, which updates the stored index, so the second `previewRedeem` (`balanceAfter`) sees a higher index than the first did. The apparent delta thus exceeds the amount, masking the slippage check (which measures the wrong delta and silently passes even when real rounding loss > 10 wei).

Example:

If interest is 1000 wei and a user deposits 500 wei, have 20 wei loss the measured `netDepositAmount` will be $1000 + 480 = 1480$ instead of 500.

Net effect:

Existing StableVault depositors lose 20 wei of yield to mask this depositor's slippage. The 10 wei tolerance was bypassed because the slippage check measured (deposit + accrual - rounding) instead of just (deposit - rounding). The rounding loss will be recorded not as interest but as `originalDeposit`.

This can be repeated every block or every time interest could have accrued in the vault but still hasn't.

The attack only profits a third party who holds strategy shares directly (could be the attacker itself), capturing their pro-rata share of the rounding sent to existing strategy holders. Profit per call $\approx (\text{index} - 1) \times \text{attackerStrategyFraction}$, bounded by the strategy's index.

Realistically the rounding is bounded by the index of the vault, so to inflict a large interest loss on the stable vault per call the index of the strategy must be large due to inflation attack (if vault is built on Solmate / a naive implementation; otherwise inflation attacks require burning meaningful assets to the virtual shares) or by being compromised and upgraded to a malicious state.

Recommendations:

Either consider not adding vaults with stale `previewRedeem` or be aware that the slippage check is weakened in such a scenario. The attacker cannot spam this attack multiple times in one block since the vault has to have unaccrued interest. Additionally we recommend monitoring vaults which are upgradable for upgrades and temporarily disabling deposits to the strategy until the upgrade is verified not to be malicious.

Customer Response:

Structurally fixed by [PR #292](#) ([fix/strategy-deposit-slippage-accrual](#)). The Allocator's `_depositToStrategy` no longer compares a stale `previewRedeem(balanceOf)` pre-snapshot against a fresh post-deposit one (which is the exact pre/post mismatch L-01 describes for strategies with a stale stored index). Instead, the Allocator snapshots the strategy-share count, runs the deposit, then computes `previewRedeem(sharesGained)` once on the post-deposit state. Result: the slippage check measures amount - `assetsRedeemableForNewShares` only, so a strategy that accrues yield during its deposit (or has a lazy `previewRedeem`) can no longer let a rounding loss > 10 wei silently pass. The recommendation footnote in L-01 (monitor upgradeable vaults / disable deposits on upgrade) remains a sensible operational practice, but the specific mask-the-slippage-check vector is closed in code.

Fix Review:

Fixed in [PR#292](#).

Informational Severity Issues

I-01: Constant `STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE` will prevent interaction with strategies once share rate hits 10

Files:

src/core/Allocator.sol

Description:

The constant `STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE` is set to 10 wei and is used to prevent value leakage on deposits into external ERC-4626 strategies. It enforces that the redeemable value of shares received is no less than the deposited amount minus 10 wei.

However, as a strategy's share rate grows over time, the rounding loss from the shares-to-assets conversion increases proportionally. Once the share rate exceeds 10, legitimate deposits may revert due to exceeding the tolerance threshold.

Example:

- Strategy share rate: 19
- Deposit amount: 10,000,000 (e.g., 10 USDC)
- Shares received: 526,315
- Redeemable asset value: 9,999,985
- Loss: 15 wei > 10 wei tolerance → deposit reverts

Recommendations:

Two alternatives were identified:

1. Retain the current constant. Accept the risk and upgrade the contract if and when the share rate of integrated strategies (e.g., USDC/USDT on Aave v3) approaches 10. This may take years or decades depending on effective interest rates.
2. Convert `STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE` to an admin-configurable parameter. This allows the tolerance to be adjusted as share rates increase, without requiring a contract upgrade.

Customer Response:

We will retain the constant with no code change, proceeding with recommendation #1 of the finding (accept the risk and upgrade if/when share rates approach the bound) rather than recommendation #2 (admin-configurable).

Key points:

1. Blast radius is manager-only after PR [#279](#). Pre-[#279](#) the constant could brick user-facing flows: `Allocator.deposit` auto-routed user deposits + bridge callbacks through `_depositToStrategy`, so a strategy reaching share rate ≥ 10 would revert at the surface. After [#279](#), `Allocator.deposit` / `depositAllowIdle` are replaced by `pullIdle(asset, amount)` which leaves funds idle. The slippage check is now only hit on manager-driven rebalance (`_allocate` $\rightarrow$ `_depositToStrategy`), so a breach reverts the rebalance only – user deposits and bridge callbacks continue to land idle. The failure mode is operationally recoverable.
2. Time-to-breach is years on the target asset set. Launch strategies are Aave V3 GHO / USDC / USDT aToken vaults. Aave V3 stablecoin indices currently sit close to 1 and grow at single-digit % APY – reaching a share rate of 10 takes many years (multiple decades at current effective rates). The Recon-Fuzz analysis in Q-18 characterises this asset class and is consistent with the years-to-decades horizon the original recommendation acknowledged.
3. An admin-configurable setter adds permanent governance surface for a parameter that needs no tuning at launch. Any setter we'd add would be ADMIN-tier + critical (matching the rest of the slippage / cap setters), so reaching for it later still goes through the timelocked admin multisig. The marginal benefit over a contract upgrade is small (skip a redeploy), and the cost is a new governance handle living on a hot contract for the entire pre-breach period.
4. The Allocator is upgradeable. When share rates approach the bound (a slow, observable trajectory – not a sudden event), an upgrade can lift the constant as part of a coordinated change with monitoring + testing, alongside any other Allocator changes that have accumulated by then. We will set up an off-chain alert on the strategy share rate well below the bound (e.g. at share rate = 5, ~half the bound) to ensure the upgrade is staged with plenty of lead time.

I-02: Transfer of Full Balance May Revert Due to Remaining Shares Redeemability Check

Files:

src/core/accounting/StableVault.sol

Description:

The transfer function enforces a `_areRemainingSharesRedeemable` check that reverts if the sender's leftover shares after the transfer are worth less than 1 wei in assets. This can cause unexpected reverts when a user attempts to transfer their full balance via `transfer()`.

Consider the following scenario:

1. A user holds a position worth exactly \$10.000000.
2. The user calls `transfer()` for exactly \$10.000000.
3. Between transaction submission and execution, the share conversion rate increases, causing the position to accrue a fraction of a wei in additional value.
4. The transaction reverts because the residual shares fail the `_areRemainingSharesRedeemable` check.

The user must instead call `transferAll()` to avoid this issue. While a valid workaround, this requires the user to identify the cause of the revert and switch to an alternative function, which degrades the user experience.

Note that `requestWithdrawal` handles this case gracefully by falling back to full withdrawal logic when remaining shares are not redeemable. The transfer function does not implement an equivalent fallback.

Recommendations:

Acknowledge as a known UX limitation. Adding a similar fallback to `transfer()` (automatically converting to a full transfer when remaining shares are not redeemable) would resolve the issue for EOA users but could introduce unexpected behavior for protocol integrations that expect partial transfers to behave deterministically.

Customer Response:

Acknowledged as a known UX limitation. A user transferring their exact full balance should use `transferAll()`; `requestWithdrawal` already falls back to full-withdrawal logic when the remaining shares are not redeemable. We deliberately did not add an equivalent auto-fallback to `transfer()`, because silently converting a partial transfer into a full one could introduce unexpected behavior for integrations that expect partial transfers to behave deterministically.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.